

Exercises

Functors and I/O

Exercise 1:

Give a definition of the function

`fmap :: (a -> b) -> IO a -> IO b`

the effect of which is to transform an interaction by applying the function to its result. You should define it using the

do

construct.

Exercise 2:

Define the function

`repeat :: IO Bool -> IO () -> IO ()`

so that

`repeat test oper`

has the effect of repeating *oper* until the condition *test* is *True*.

Exercise 3:

Define the higher-order function *whileG* in which the condition and the operation work over values of type *a*. Its type should be:

`whileG :: (a -> IO Bool) -> (a -> IO a) -> (a -> IO a)`

so that

`whileG cond op x`

has the effect of repeating *op x* while the condition *cond x* is *True*.

Exercise 4:

Using the function *whileG* or otherwise, define an interaction which reads a number, *n*, say, and then reads a further *n* numbers and finally returns their average.

Exercise 5:

1. Define a function

```
accumulate :: [IO a] -> IO [a]
```

which performs a sequence of actions and accumulates their result in a list.

You can test this using (see testf below), e.g.:

```
> accumulate [readLn, testf "hi", readLn]
```

2. Also define a function :

```
sequence' :: [IO a] -> IO ()
```

which performs the interactions in turn, but discards their results.

You can test this using (see testf below), e.g.:

```
> sequence' [putStrLn "hello" , putStrLn "goodbye"]
```

3. Finally show how you would sequence a series, passing in values from one to the next :

```
seqList :: [a -> IO a] -> a -> IO a
```

Hint: Use a simple function e.g

```
testf :: String -> IO String  
testf x = do  
           putStrLn x  
           return (x ++ x)
```

(which takes a parameter and appends it to itself. It works on Strings).

So, you could call it as

```
> seqList [testf, testf, testf] "hello"    — and get back  
hello                                         —from first call (as IO effect)  
hellohello                                   —from second call  
hellohellohellohello                        —from third call  
"hellohellohellohellohellohellohellohello" — returned from function
```

Exercise 6:

Given the type definition

```
data Result a = Succeed a | Fail
```

show how **Result** can be made into a monadic type.